

Building AI Applications with Foundation Models

**A Beginner's Guide to Large Language Models, LangChain,
and Retrieval-Augmented Generation**

Suman Samui

Preface

This book grew out of a course for students who can already write Python but have not yet worked with modern language models. The aim is narrow and practical: to explain how today's AI applications are actually put together, and to do so from the ground up, without assuming a background in machine learning.

The organizing idea is a single running project. Rather than survey the field, we build one thing and improve it in stages: a personal assistant that can hold a conversation, remember what has been said, and answer questions about documents that its underlying model has never seen. Each chapter adds one capability to this assistant and explains the concept behind it. By the end the reader will have seen, end to end, how a retrieval-augmented question-answering system works, and will understand each part well enough to change it.

The treatment is deliberately conceptual. Where a diagram or a short piece of code clarifies an idea, it is included; where prose does the job better, code is left out. The intent is that a reader should be able to follow the argument on a train, away from a keyboard, and still come away knowing what an embedding is, why chunking matters, and what a language model can and cannot be trusted to do.

A word on tone. The subject attracts a great deal of excitable language, and some of the claims made about these systems do not survive contact with a real project. I have tried to describe the technology as it behaves rather than as it is marketed. A language model is a useful and often surprising tool, but it is a tool with definite failure modes, and a working engineer is served better by understanding those failure modes than by admiring the successes.

The three parts of the book follow the arc of the course. Part I sets out what foundation models are and how one talks to them. Part II introduces LangChain, the framework we use to assemble the pieces. Part III addresses the central limitation of a language model, that it does not know your documents, and develops retrieval-augmented generation as the standard answer to it. A closing chapter steps back to consider what it means to work as an AI application engineer.

No prior exposure to neural networks is required. The small amount of mathematics that appears is confined to a few ideas, chiefly probability over sequences and the geometry of vectors, and each symbol is defined where it is introduced. Readers who prefer to skip the equations on a first pass can do so without losing the thread.

To the Instructor

The book is written to be taught, and this short section sets out the teaching order it assumes so that an instructor can follow it, depart from it deliberately, or hand parts of it to students to read on their own.

One arc, used throughout

Most chapters follow the same path from a standing start to working knowledge. A concept is introduced by the problem it solves, made intuitive before it is made precise, stated plainly, given whatever mathematics it genuinely needs, shown in a concrete example, interpreted so that its meaning and limits are clear, and finally handed to the student to practice. In shorthand:

Motivation $\rightarrow$ Intuition $\rightarrow$ Concept $\rightarrow$ Mathematics $\rightarrow$ Example $\rightarrow$ Interpretation $\rightarrow$ Practice

The order is a default, not a rule. Some chapters carry their motivation over from the one before and open directly on the concept; a few, whose job is to frame a problem rather than solve it, end on interpretation with the solution deferred to the next chapter; and the chapters with little mathematics simply omit that step. Where the material reads more naturally in another order, the book takes the other order. An instructor pressed for time can treat the mathematics step as optional throughout: the equations are confined to a few ideas and are set up so that the surrounding prose stands without them.

A single project, growing

What binds the chapters is not only this shared shape but a shared object. The book builds one assistant and improves it in stages, and each chapter adds a capability to it. That construction runs the length of the book: an assistant that at first can only answer a single question learns to be reached through a framework, to take shaped instructions, to remember a conversation, to be assembled as a clean pipeline, to emit structured data, and at last to answer from documents it was never trained on. Keeping the running project in view is the surest way to stop the individual techniques from feeling like a disconnected list, and it helps to remind students at the start of each part what the assistant can already do and what the coming chapters will let it do next.

The three parts group the chapters by role. Part I establishes what the model is and how to reach it; Part II builds the application around it with LangChain; Part III solves the knowledge problem with retrieval and completes the assistant. A short introduction opens each part and restates where the project stands.

Notation

The conventions below are used throughout. They are collected here for reference; every symbol is also defined at the point where it first appears.

Symbol	Meaning
a, b, d, k, n, t	scalars (plain italic lowercase)
$\mathbf{x}, \mathbf{q}, \mathbf{c}$	vectors (bold lowercase)
$\mathbf{W}$	a matrix (bold uppercase)
$\mathcal{V}$	a set; here, the vocabulary of tokens (calligraphic)
$ \mathcal{V} $	the number of elements in the set $\mathcal{V}$
$\mathbb{R}^d$	the space of real vectors with d components
$x_{1:t}$	the sequence of tokens $x_1, x_2, \dots, x_t$
$p(\cdot)$	a probability (or probability distribution)
$p(a \mid b)$	the probability of a given b
θ	the trainable parameters of a model
f_θ	a function (a neural network) with parameters θ
$\langle \mathbf{a}, \mathbf{b} \rangle$	the inner (dot) product of $\mathbf{a}$ and $\mathbf{b}$
$\ \mathbf{a}\ $	the Euclidean length of the vector $\mathbf{a}$
$\text{sim}(\cdot, \cdot)$	a similarity score between two vectors
$\text{softmax}(\cdot)$	the softmax function, defined in Chapter 2
TopK	selection of the k items with the largest score
τ	the sampling temperature

Indices such as i, j , and t run over the natural numbers and their range is stated wherever a sum or a sequence appears. Model parameters (θ) are distinguished from user-chosen settings, called hyperparameters, such as the chunk length or the number of retrieved passages k ; the latter are fixed by the engineer rather than learned from data.

Contents

Preface	ii
To the Instructor	iii
Notation	iv
I Foundations	1
1 AI as a General-Purpose Technology	3
1.1 What a general-purpose technology is	3
1.2 The effect on ordinary work	3
1.3 A tool that is right on average	4
1.4 Three ways to work with AI	4
1.5 What an application engineer needs	5
2 Foundation Models and Large Language Models	7
2.1 The 2022 shift	7
2.2 What a foundation model is	7
2.3 What a language model actually does	8
2.4 Why the model is a stochastic tool	9
2.5 The families in brief	10
3 Talking to a Language Model	11
3.1 Why conversation was hard before	11
3.2 The assistant we are going to build	11
3.3 Two ways to run a model	12
II Building with LangChain	14
4 A Framework for Building with Models	16
4.1 What the model does not give you	16
4.2 Setting up a project	16
4.3 Reaching the model from Python	17
5 Prompts and Prompt Templates	19
5.1 From a fixed prompt to a reusable one	19
5.2 PromptTemplate	19
5.3 Chat prompts: separating the roles	20

6	Conversation and Memory	22
6.1	The model forgets between calls	22
6.2	Short-term memory by resending the history	22
6.3	What this memory can and cannot do	23
6.4	Memory does not cure hallucination	23
7	Chains	25
7.1	Components in a line	25
7.2	Connecting components with LCEL	25
7.3	Why this is more than convenience	26
8	Runnables: The Common Interface	27
8.1	One interface, three verbs	27
8.2	You have been using Runnables already	27
8.3	Turning ordinary functions into Runnables	28
8.4	Chains that are not straight lines	28
9	From Chatbots to Agents	30
9.1	Talking versus acting	30
9.2	Software needs structure	30
9.3	Two kinds of output	30
9.4	Getting structure out of the model	31
III	Retrieval-Augmented Generation	33
10	The Knowledge Problem	35
10.1	A model does not know your documents	35
10.2	Two tempting non-solutions	35
10.3	What is actually needed	36
11	Retrieval-Augmented Generation	37
11.1	What RAG is, and is not	37
11.2	Two phases	37
11.3	Where the difficulty lies	38
12	Embeddings and Semantic Similarity	40
12.1	Meaning as a vector	40
12.2	Words versus sentences	40
12.3	Measuring closeness by angle	41
12.4	Putting the two together	42
13	Chunking and Vector Stores	43
13.1	Why documents are cut into chunks	43
13.2	Overlap	43

13.3 Storing embeddings for fast search	44
14 Case Study: Chatting with a Video	46
14.1 The problem	46
14.2 The solution in outline	46
14.3 Getting the text out of the video	46
14.4 Indexing the transcript	47
14.5 Answering a question	47
14.6 What has been built	49
A Closing Note: Where to Go Next	50

Part I

Foundations

This part builds the picture the rest of the book depends on. It explains why a language model is worth learning, what such a model actually is and how it fails, and how to obtain an answer from one. The three chapters move from the general case for the technology, to the mechanism inside the model, to the practical act of running it. By the end the running assistant can do one modest thing, take a question and return an answer, either from a hosted model or from one running privately on your own machine. It cannot yet remember, and it cannot yet be trusted on anything specific; those are the problems the later parts take up. What matters here is leaving with an accurate, unromantic sense of the component everything else is built around.

Chapter 1

AI as a General-Purpose Technology

A reasonable first question for anyone starting a course like this is whether the effort is worth it. Software already does a great deal; why should a working engineer, or a student about to become one, spend time on yet another set of libraries? The honest answer is that the systems built on large language models are not simply another library. They belong to a small category of technologies whose influence spreads across many unrelated fields, and that spread changes what it is useful to know.

1.1 What a general-purpose technology is

Economists use the term *general-purpose technology* for inventions whose effect is felt far beyond the industry that produced them. Electricity is the standard example. It was not confined to lighting; once cheap electric power was available, factories were rebuilt around it, homes were rewired for it, and entirely new products, from the refrigerator to the computer, became possible. The printing press and the internal combustion engine belong to the same category. What these have in common is that they are enabling rather than specialized. They do not solve one problem well; they lower the cost of solving a whole class of problems, and other people then find uses their inventors never imagined.

Modern AI has begun to behave in this way. A single model, trained once, can draft an email, translate a contract, summarize a medical report, write a first version of a program, and answer questions about a photograph. None of these was the model's specific purpose, and that is exactly the point. The capability is broad, and it is being applied in law, medicine, finance, design, teaching, and manufacturing by people who did not build the model and often do not understand its internals. When a technology is used this widely by people outside its field of origin, the ability to work with it stops being a specialist skill and starts to resemble literacy.

1.2 The effect on ordinary work

It is easy to overstate this, and much of the public discussion does. A more measured observation is that these tools are changing the texture of ordinary professional work rather than replacing whole professions overnight. A lawyer still decides what advice to give, but the first draft of a memorandum may now come from a model. A programmer still designs the system, but a good deal of routine code is generated and then corrected. The work shifts from producing every artifact by hand toward directing, checking, and repairing the output of a fast but unreliable assistant.

This shift has a name in casual use. Writing programs by describing what you want in

ordinary language, letting a model produce the code, and then iterating on the result has come to be called *vibe coding*. The phrase is loose, and it can encourage sloppy habits, but it captures something real: for a growing set of tasks the bottleneck is no longer typing the code but knowing what to ask for and being able to judge whether the answer is correct. That judgment is where the value now sits, and it is not something the tool provides.

The economic argument that accompanies all of this is blunt. A software agent that performs a task does not tire, does not need to sleep, and, once built, costs a small fraction of a salaried person to run. Where a task can be handed to such an agent, the pressure to do so is strong. Whether this is good news depends entirely on where one sits, but the direction of travel is not seriously in doubt, and it is a poor strategy to be on the side of the work that is easiest to automate.

1.3 A tool that is right on average

Before going further, one property of these systems has to be stated precisely, because misunderstanding it is the source of most disappointment. A language model is a *stochastic* tool. Given the same question twice it may give two different answers, and either answer may be wrong while sounding entirely confident. This is not a defect to be patched; it follows from how the model works, as the next chapter explains. The model produces plausible text, and plausible is not the same as correct.

The practical consequence is that a language model should be treated less like a calculator and more like a knowledgeable but occasionally mistaken colleague. A calculator that returned the wrong product one time in fifty would be thrown away. A colleague who is usually right, fast, and cheap, but who must be checked on anything that matters, is still extremely useful, provided one never forgets the checking. Almost every sound engineering pattern in this book exists to manage that gap between usually right and reliably right. Retrieval, which occupies the third part of the book, is the clearest example: it exists so that the model answers from documents you trust rather than from its own uncertain memory.

1.4 Three ways to work with AI

People who work with AI do so at very different depths, and it helps to be clear about which one this book is aimed at. Three broad levels are useful to distinguish, illustrated in Figure 1.1.

At the first level are the *application users*. They open a finished product, type a question, and read the answer. Fluency here is a matter of asking good questions and knowing what the tool is for; no programming is involved. This is already a valuable skill and it is the level most people will reach.

At the second level are the *application developers*. They take models that someone else has trained and assemble them, together with other software, into new applications: a support assistant that draws on a company's manuals, a tool that reads invoices, an interface that lets a user chat with a video. This is the work of an AI application engineer, and it is the subject of this book.

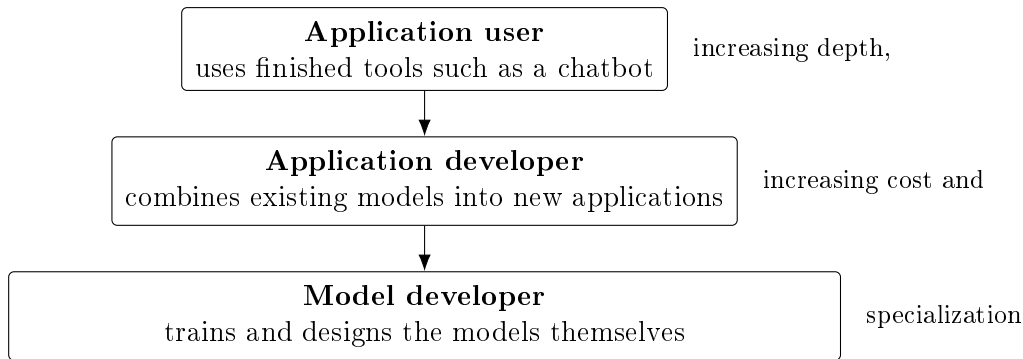


Figure 1.1: Three levels at which one can work with AI. This book is concerned with the middle level: building applications from models that already exist.

At the third level are the *model developers*. They design the network architectures, gather and clean enormous datasets, and run the training that produces a model in the first place. This is demanding, resource-intensive work carried out by a comparatively small number of teams. Understanding it in outline is useful, and Chapter 2 provides that outline, but the course does not attempt to train a model from scratch. There is no need to. The central fact that makes application engineering possible is that capable models already exist and can be used as components.

1.5 What an application engineer needs

The engineer at the middle level does not need to be able to derive the training algorithm, but does need a working command of several things. The first is a correct mental model of what a language model is and how it fails, so that the application is designed around its limits rather than in denial of them. The second is fluency with the ordinary tools of software: the ability to write clean Python, manage dependencies, call an interface over the network, and handle data. The third is familiarity with the specific patterns that have emerged for putting models to work, chiefly prompting, chaining, memory, tool use, and retrieval, which are the patterns this book develops one by one.

None of this requires a research background. It requires the ordinary discipline of an engineer: build the smallest thing that works, test it against reality, and improve it. The rest of the book is an extended exercise in exactly that discipline, applied to a single project that grows chapter by chapter into a document-aware assistant.

Exercises

1. Name two general-purpose technologies other than those discussed in this chapter and, for each, give one industry it affected that is far from its origin.
2. The chapter argues that a language model should be checked “on anything that matters.” Describe a task where you would trust an unchecked model answer, and one where you would not, and state the property that distinguishes them.
3. Which of the three levels of AI work best describes each of the following: a doctor using a

symptom-checker app; a team fine-tuning a model on legal text; an engineer building a chatbot over a company's manuals?

Chapter 2

Foundation Models and Large Language Models

The previous chapter described what these systems do from the outside. This one looks inside far enough to build a correct intuition, and no further. The reward for reading it carefully is that the failure modes discussed later, hallucination in particular, will seem obvious rather than mysterious.

2.1 The 2022 shift

For most of the history of applied machine learning, a model was built for one task. A spam filter classified email; a translation system converted French to English; a speech recognizer turned audio into text. Each was trained on data specific to its job and was useless outside it. Building a new capability meant collecting a new dataset and training a new model, which was slow and expensive.

The public release of capable conversational models toward the end of 2022 marked a change in this pattern. What reached ordinary users was a single model that could translate, summarize, answer questions, write code, and hold a conversation, none of which had been programmed as a separate feature. The same model, without retraining, handled tasks it had never been shown explicitly. The interesting shift was not that the model was larger, though it was, but that one model now covered a broad range of tasks that had previously each required their own system.

2.2 What a foundation model is

The models that make this possible are called *foundation models*, and the name is apt: they are meant to serve as a base on which many applications are built. A foundation model can be described by three properties that must hold together.

It is, first, a large neural network. The architecture, almost always a variant of the transformer, has a great many adjustable numbers, called parameters and written collectively as θ . It is trained, second, on a very large and broad body of data rather than on material for a single task. And it is designed, third, to be adapted afterward to many specific uses rather than to perform one fixed job. Remove any of the three and the description no longer fits: a large network trained narrowly is just a big special-purpose model, and a small network is not a foundation model however broad its training data.

A *large language model*, or LLM, is a foundation model whose data and purpose are language. It reads and produces text. When such a model also handles images, audio, or video alongside text, it is called *multimodal*, and a generative multimodal model is sometimes called a large multimodal model. The distinction is one of inputs and outputs rather than of kind: a text-only

model predicts what comes next given text, while a multimodal model predicts what comes next given text together with, say, the contents of an image. The mechanism underneath is the same, and it repays a closer look on its own.

2.3 What a language model actually does

Underneath the conversation, an LLM performs a single, narrow operation over and over: it predicts the next token. A *token* is a small unit of text, roughly a short word or a piece of one, and the full set of tokens the model knows is its vocabulary, written $\mathcal{V}$. A stretch of text is a sequence of tokens $x_1, x_2, \dots$, which we abbreviate $x_{1:t}$ for the first t of them.

Given the tokens seen so far, the model assigns a probability to every possible next token. Writing $x_{1:t-1}$ for the context and x_t for the token to be predicted, the model computes

$$p(x_t \mid x_{1:t-1}), \quad (2.1)$$

a probability distribution over the whole vocabulary $\mathcal{V}$. To generate text, the model draws a token from this distribution, appends it to the context, and repeats (Figure 2.1). Producing a paragraph is nothing more than doing this a few hundred times in a row. The probability that the model assigns to a complete sequence of length T follows from the chain rule of probability,

$$p(x_{1:T}) = \prod_{t=1}^T p(x_t \mid x_{1:t-1}), \quad (2.2)$$

which simply says that the probability of the whole text is the product of the probabilities of each token given everything before it.

The distribution in (2.1) is produced from the model’s parameters in two stages. The network f_θ maps the context to a vector of raw scores, one per vocabulary item,

$$\mathbf{z} = f_\theta(x_{1:t-1}) \in \mathbb{R}^{|\mathcal{V}|},$$

and these scores, called *logits*, are turned into probabilities by the softmax function. For a candidate token $v \in \mathcal{V}$,

$$p(x_t = v \mid x_{1:t-1}) = \text{softmax}(\mathbf{z})_v = \frac{\exp(z_v)}{\sum_{u \in \mathcal{V}} \exp(z_u)}. \quad (2.3)$$

Each logit is exponentiated so that it is positive, and the results are divided by their total so that they sum to one, which makes them a valid probability distribution over the vocabulary. A large logit becomes a large probability; the token the model considers most likely is the one with the highest score.

The parameters θ are set during training so that the probabilities the model assigns match the patterns in an enormous body of text. Nothing in this procedure gives the model a notion of truth. It learns statistical regularities of language, which tokens tend to follow which, across a scale of data no person could read in many lifetimes, and it stores those regularities in its

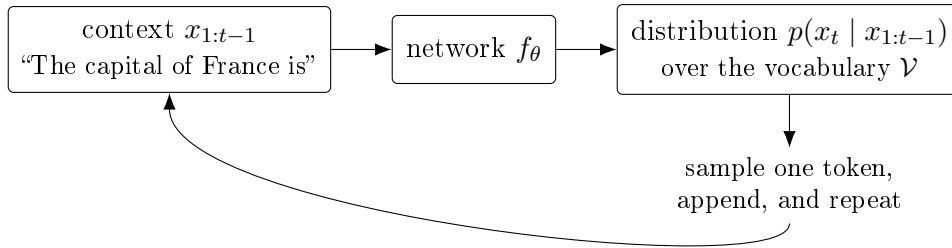


Figure 2.1: A language model generates text one token at a time. The context is mapped to a probability distribution over the vocabulary; a token is drawn from it, added to the context, and the process repeats.

parameters. That is the whole of it, and it explains a great deal.

2.4 Why the model is a stochastic tool

Because the model draws the next token from a distribution rather than picking a fixed answer, its output varies. How much it varies is controlled by a single setting, the *temperature* $\tau > 0$, applied to the logits before the softmax:

$$p_{\tau}(x_t = v \mid x_{1:t-1}) = \frac{\exp(z_v/\tau)}{\sum_{u \in \mathcal{V}} \exp(z_u/\tau)}. \quad (2.4)$$

As τ approaches zero the distribution concentrates on the single highest-scoring token, and generation becomes nearly deterministic. As τ grows the distribution flattens and the output becomes more varied and less predictable. A low temperature suits tasks with one right answer; a higher temperature suits tasks, such as brainstorming, where variety is wanted. Setting τ is one of the few knobs an application engineer routinely turns.

The same mechanism explains the model’s central weakness. The training objective rewards text that is probable, meaning text that resembles what people have written, not text that is true. When the context concerns something the training data covered well, probable and true usually coincide, and the model appears knowledgeable. When the context concerns something specific, private, or recent that the training data did not cover, the model still produces fluent, probable-sounding text, but there is nothing tying that text to fact. The result is a confident, well-formed, wrong answer. This is what people mean by *hallucination*. It is not the model breaking; it is the model doing exactly what it was built to do, which is to continue the text plausibly.

A well-worn comparison is that an LLM is like a parrot that reproduces the patterns of speech without grasping their meaning. The comparison is a useful corrective to the opposite error, that of treating the model as an oracle, and one to keep in mind whenever the output is impressive. It should not be pushed too far. The regularities these models capture are rich enough to support genuinely useful behavior, and dismissing them as mere mimicry underestimates what can be built with them. The sound position is in between: the model has no access to truth, yet the statistical structure it has absorbed is powerful, and the engineer’s task is to supply the truth

Category	What it takes in and produces
Language model	text in, text out
Multimodal model	text and images (or audio) in, text out
Image-generation model	text in, an image out
Speech model	audio in, text out, or the reverse

Table 2.1: Common families of foundation model, organized by modality. The underlying next-token mechanism is shared; the families differ in what they read and what they write.

the model lacks.

2.5 The families in brief

Foundation models now come in several families, distinguished by the modalities they handle rather than by any deep difference in principle. Table 2.1 lists the common categories. The reader need not memorize particular product names, which change quickly; what matters is the organizing idea, that a small number of broadly trained models now serve as the raw material for a wide range of applications.

Two points from this chapter carry into the rest of the book. A foundation model is a large model trained on broad data that can be adapted to many tasks, and an LLM is such a model specialized to language. The assistant we build uses an LLM as one component among several. It is a component with a specific character: fluent, broadly capable, fast, cheap, and unreliable on anything outside its training. Every later chapter can be read as a way of getting useful work out of a component with exactly that character.

Exercises

1. Using (2.2), explain in words why a model that predicts each next token well will also assign high probability to sensible whole sentences.
2. Show from (2.4) that as $\tau \rightarrow 0$ the model almost always selects the token with the largest logit. What happens to the spread of outputs as τ becomes large?
3. A colleague says the model “looked up” the answer to a factual question. Explain why this description is inaccurate, referring to what the softmax in (2.3) actually computes.
4. Give one task for which you would set a low temperature and one for which you would set a high temperature, and justify each choice.

Chapter 3

Talking to a Language Model

Having a model is one thing; getting an answer out of it is another. The act of running a trained model on an input to obtain an output is called *inference*, and this chapter covers the two ordinary ways to do it, the trade-off between them, and the goal that the rest of the book works toward.

3.1 Why conversation was hard before

It is easy to forget how difficult a decent chatbot was to build only a few years ago. The assistants of that era were, for the most part, rule-based. An engineer wrote out the conversation in advance as a set of branches: if the user says this, reply with that; if the user asks about billing, follow the billing script. Such a system works only as long as the user stays on a path the engineer anticipated, and users almost never do.

Three problems defeated the rule-based approach. The first was the need for flexibility. Real conversations wander, double back, and combine topics, and no fixed set of branches can cover the possibilities; a rigid script feels rigid the moment the user steps off it. The second was understanding language as people actually use it, with typos, slang, abbreviations, and countless ways of phrasing the same request. Matching keywords is brittle: a system keyed to the word “refund” misses “money back,” “return my payment,” and “I want it undone.” The third was memory. A useful assistant should recall what was said earlier in the conversation and adjust, which a stateless script cannot do.

Language models dissolve the first two problems almost incidentally. Because a model produces the next token from context rather than following a script, it handles unforeseen turns without special handling, and because it was trained on an enormous range of real text, it copes with informal and varied phrasing as a matter of course. The rule-based era gave way, first to models that could understand language, and then, from 2022, to conversational LLMs capable enough to hold a general discussion. The third problem, memory, is not solved for free, and Chapter 6 is devoted to it.

3.2 The assistant we are going to build

It helps to fix the target now. Across the book we build a single assistant with four properties, chosen because together they exercise everything worth learning:

- it holds a conversation and remembers what has been said within it;
- it runs on an open-source model that can be hosted locally, so that private data need not leave the machine;

	Hosted (API)	Local (e.g. Ollama)
Capability	very large models available	smaller models
Cost	billed per call	no per-call charge
Hardware	none needed	adequate memory needed
Privacy	data sent to a third party	data stays on the machine
Setup	an account and a key	a one-time download

Table 3.1: The two common routes to inference. The application code differs only slightly between them, which is what makes it easy to start with one and move to the other.

- it can consult documents the user provides and answer questions about them; and
- it declines to invent an answer when the documents do not contain one, rather than hallucinating.

The last two properties are the hard ones, and they are the reason retrieval occupies the final part of the book. The first two we can begin on immediately, once we can perform inference at all.

3.3 Two ways to run a model

There are two common ways to obtain an answer from an LLM, and the choice between them shapes the rest of an application.

The first is to call a model that runs on someone else’s servers through an *application programming interface*, or API. The application sends the input over the network and receives the generated text in reply. This gives access to very large, very capable models without owning any special hardware, and it is the quickest way to start. The costs are that each call is billed, that the data in each request leaves the machine and is sent to a third party, and that the application depends on that party’s continued service.

The second is to run a model *locally*, on your own machine. Open-source models can be downloaded and served on ordinary hardware, and a tool such as Ollama makes this routine: it downloads a chosen model and exposes a small local server that the application calls exactly as it would call a remote one. The models one can run locally are typically smaller than the largest hosted models, and running them requires adequate memory, but the data never leaves the machine and there is no per-call charge. For an assistant that is meant to handle private documents, the privacy of local inference is not a minor convenience; it is often the deciding factor.

The pattern is the same either way. The application prepares an input, sends it to a model, and receives generated text. Because the shape of the interaction is identical, an application can be written against a local model during development and pointed at a hosted one later, or the reverse, with little change. The next listing shows the bare form of a local call through Ollama; a hosted call differs only in the client that is constructed and the credentials it carries.

```
import ollama
```

```
response = ollama.chat(  
    model="llama3",  
    messages=[{"role": "user", "content": "Explain inference in one sentence."}],  
)  
print(response["message"]["content"])
```

Listing 3.1: The minimal shape of a local inference call. The details of the client library matter less than the pattern: send a message, receive generated text.

This is already a working, if trivial, program: it sends a single message and prints the model's reply. Everything that follows is a matter of surrounding this core with the machinery a real assistant needs, a way to shape the instructions, a way to keep the conversation, a way to reach documents, and a way to combine these cleanly. Assembling that machinery by hand quickly becomes tedious, which is why the next part of the book introduces a framework built for the job.

Exercises

1. Give two reasons a rule-based assistant fails on real conversations, and explain why a language model avoids each without special handling.
2. For each of the following, state whether hosted or local inference is the better fit and why: a public demo with no budget; an assistant that reads confidential contracts; a prototype that needs the strongest available model for one week.
3. The assistant's fourth property is that it should refuse to invent answers. Why is this property difficult to guarantee given what Chapter 2 established about how a model generates text?

Part II

Building with LangChain

With a way to reach a model in hand, this part turns a single call into an application. It introduces LangChain, the framework used for the rest of the book, and then develops the patterns an assistant is made of, one per chapter: shaping the model's input with prompts, giving it memory of the conversation, joining components into chains, understanding the common interface that lets those components compose, and finally producing structured output so the model can drive software rather than only talk. Each chapter adds a capability to the same assistant, and by the end it holds a coherent conversation and is assembled as a clean pipeline whose parts can be swapped without disturbing the whole. One limitation survives everything here, and it is deliberate: the assistant still answers from the model's own memory, which means it still cannot be trusted on your documents. Removing that limitation is the work of Part III.

Chapter 4

A Framework for Building with Models

The single call at the end of the last chapter is the atom of an application, but an application is not one atom. A useful assistant has to combine several ingredients around the model, and doing so by hand leads to repetitive, brittle code. A framework exists to handle the repetition. The one used throughout this book is LangChain, described by its author, Harrison Chase, as a framework for developing applications powered by language models.

4.1 What the model does not give you

A raw model call takes some text and returns some text. A working chatbot needs rather more than that, and the list of what is effectively the syllabus for this part of the book.

A practical assistant usually needs a *system instruction* that fixes its role and manner once, so that the behavior is consistent across a whole conversation rather than restated in every message. It needs a record of the *conversation history*, so that a later question can refer to an earlier answer. It often needs *access to documents*, so that it can answer from material the model was never trained on. And it increasingly needs the ability to *call external tools*, so that it can look something up, run a calculation, or take an action rather than only produce text. Figure 4.1 shows these arranged around the model at the center.

None of these is conceptually hard. The difficulty is in the plumbing: passing the right text to the model in the right shape, threading history through every call, formatting retrieved documents, parsing what comes back. A framework earns its place by making that plumbing uniform, so that swapping one model for another, or adding a step to a pipeline, does not require rewriting everything downstream. LangChain does this by giving every component a common interface, a design we return to in Chapter 8 once there are enough components to make the payoff visible.

4.2 Setting up a project

Before writing any code it pays to put the project on a sound footing, because dependency problems cause more wasted hours than any bug in the application itself. Two habits prevent most of that waste: keep each project's dependencies separate from every other project's, and record exactly which versions the project uses so that it can be reproduced later or on another machine.

The tool this course uses for both is `uv`, a modern Python package and project manager from Astral. It creates an isolated environment for the project, installs packages into it quickly, and keeps a record of what was installed. Using it is a matter of a few commands: initialize a project,

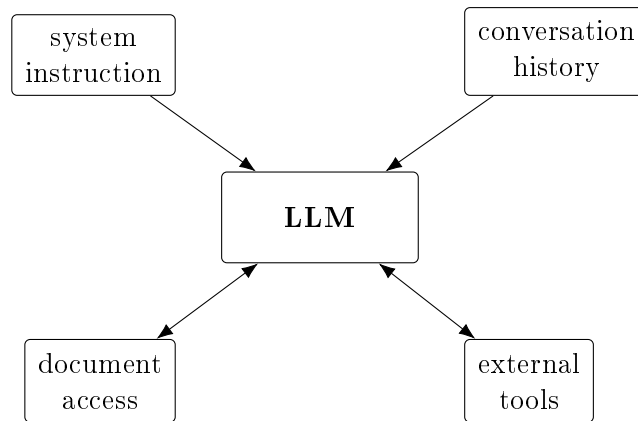


Figure 4.1: A model produces text; an application supplies everything around it. LangChain provides a uniform way to attach these ingredients to a model and to connect them to one another.

add the packages the application needs, and run the code inside the managed environment. The specific commands are documented and change over time, so they are not reproduced here; the point to absorb is the discipline. A project that declares its dependencies explicitly can be handed to a colleague and will behave the same way for them, which a pile of loose scripts cannot promise.

4.3 Reaching the model from Python

With an environment in place, the application needs a way to send text to a model and read the reply, now through LangChain rather than a model-specific client. LangChain wraps each provider, whether a hosted API or a local server such as Ollama, in a common object usually called a chat model. Once such an object is constructed, the application interacts with it the same way regardless of which model sits behind it.

```

from langchain_ollama import ChatOllama

model = ChatOllama(model="llama3")
answer = model.invoke("Explain a foundation model in one sentence.")
print(answer.content)

```

Listing 4.1: Reaching a model through LangChain. The provider-specific detail is confined to the construction of the model object; the call that follows is uniform.

The method that runs the model is called `invoke`. It takes an input, runs the model, and returns a result. That name will recur: as the next chapters will show, `invoke` is not special to the model but is the shared way of running *every* LangChain component, from a prompt to a whole pipeline. Establishing it here, on the simplest possible case, is deliberate. By the end of Part II the same verb will run a chain of a dozen steps, and it will help to remember that it started as nothing more than a way to ask a model a question.

Exercises

1. List the four ingredients a practical assistant needs around the model, and for each give a one-line example of a conversation in which its absence would be felt.
2. Explain, to someone who writes scripts without a project manager, one concrete problem that isolating a project's dependencies prevents.
3. The chapter claims that **invoke** will later run an entire pipeline, not just a model. Why is it useful for a framework to give unrelated components the same method name?

Chapter 5

Prompts and Prompt Templates

The input given to a language model is called a *prompt*. It is the text, or the set of messages, sent to the model to guide what it produces. Since the model conditions its output entirely on this input, the prompt is the main lever an application has over the model's behavior, and learning to construct prompts well is a large part of the craft.

5.1 From a fixed prompt to a reusable one

The simplest prompt is a fixed string. One writes out the full instruction, sends it, and reads the answer. This is fine for a task that is done once and never varies, a standalone run whose input is known in advance. It is a poor fit for an interactive application, where the instruction is the same each time but part of it, the user's actual question, is different on every call.

Consider a tutor that explains a topic in a beginner-friendly way. The surrounding instruction never changes: explain the following topic simply, keep it short, give the main idea and an application. Only the topic changes from one use to the next. Writing the whole prompt out afresh for each topic, by pasting the topic into a long string, is exactly the kind of repetition a framework should remove. What is wanted is a fixed pattern with a labeled gap, into which the varying part is dropped when the application runs. That is a *prompt template*.

5.2 PromptTemplate

A `PromptTemplate` is a reusable string prompt that contains one or more variables, filled in at the moment the application runs. The fixed text is written once; the variable positions, the *placeholders*, are marked by name. LangChain's own description is a string template that accepts parameters and formats a prompt for a language model.

```
from langchain_core.prompts import PromptTemplate

template = PromptTemplate.from_template(
    "Explain the topic '{topic}' to a beginner in two sentences."
)
prompt = template.invoke({"topic": "foundation models"})
```

Listing 5.1: A prompt template separates the fixed instruction from the varying input. The placeholder `topic` is supplied when the template is invoked.

The gain is not merely tidiness. Because the instruction lives in one place, it can be revised, tested, and reused without touching the rest of the application, and the same template can serve every topic a user might ask about. The template is filled by the same `invoke` verb introduced

in the previous chapter, which is the first sign that prompts and models are meant to fit together into a single pipeline.

5.3 Chat prompts: separating the roles

Modern conversational models do not see one undifferentiated block of text. They see a list of messages, each tagged with a *role*. Two roles matter at this stage. A *system* message sets the assistant’s standing instructions, its role, tone, and constraints, and it applies to the whole conversation. A *human* message carries what the user actually says. Keeping these apart matters, because the two are meant to be treated differently: the system message is the assistant’s brief, and the human messages are the traffic of the conversation.

A `ChatPromptTemplate` builds this structured, multi-message prompt rather than a single string. It lets the engineer specify the system message and the human message separately, each of which may contain placeholders of its own.

```
from langchain_core.prompts import ChatPromptTemplate

chat_template = ChatPromptTemplate.from_messages([
    ("system", "You are an experienced research mentor. "
              "Explain clearly and keep answers short."),
    ("human", "Explain the research topic '{topic}' "
              "in a single beginner-friendly paragraph."),
])
messages = chat_template.invoke({"topic": "foundation models"})
```

Listing 5.2: A chat prompt template produces separate system and human messages, each of which may carry placeholders.

Invoking this template yields two messages: a system message carrying the mentor instruction and a human message carrying the request with the topic filled in. If a template contains no placeholders, it is still a valid chat prompt, but a static one: it produces the same messages every time and cannot adapt to input. The reason to reach for a template at all is the placeholder, which is what turns a fixed instruction into a *dynamic* prompt, one whose content depends on what the user supplies at runtime.

This distinction, static versus dynamic, is the whole content of the chapter. A static prompt is adequate for a one-off task. An interactive assistant needs dynamic prompts, and a chat prompt template is the ordinary way to produce them: a fixed frame that states who the assistant is, with gaps for what the user wants. With prompts in hand, the next question is how the assistant remembers what has already been said, which is the subject of the following chapter.

Exercises

1. Rewrite the tutor instruction from this chapter as a single fixed string for the topic “embeddings,” then as a `PromptTemplate`. State one maintenance advantage of the second form.
2. Explain why a conversational model is given messages tagged with roles rather than one block of text, and what the system message is for.

3. A template with no placeholders is described as “static.” Give an example of a genuinely useful static chat prompt, and explain why it does not need to be dynamic.

Chapter 6

Conversation and Memory

A model call is stateless. The model reads its input, produces an output, and retains nothing; the next call begins with no knowledge of the last. This is easy to forget, because a conversation with a good assistant feels continuous, as though it remembers. It does not, at least not on its own. The continuity is something the application constructs, and understanding how is the point of this chapter.

6.1 The model forgets between calls

Suppose a user tells the assistant their name, and a few turns later asks the assistant to repeat it. If each turn is a separate, independent call, the second call has no way to know the answer, because the name was given in a different call that left no trace. The model is not being obtuse; it genuinely has no record. Whatever memory the assistant appears to have must be supplied from outside the model.

6.2 Short-term memory by resending the history

The remedy is direct. If the model only knows what is in the current input, then put the history *into* the current input. Before each new call, the application assembles the full list of messages exchanged so far, the system instruction, then every human and assistant message in order, then the new question, and sends the whole list. The model reads the accumulated conversation as context and answers in light of it. Repeating this on every turn produces what is called *short-term conversational memory*: memory that lasts as long as the application keeps resending the growing list. Figure 6.1 shows the arrangement.

In code this is a list that grows. Each user message is appended to it, the whole list is sent, and the reply is appended in turn, so that the next call sees everything that came before.

```
history = [("system", "You are a helpful assistant.")]

while True:
    user_text = input("You: ")
    if user_text == "exit":
        break
    history.append(("human", user_text))
    reply = model.invoke(history)      # the whole history is the input
    print("AI:", reply.content)
    history.append(("ai", reply.content))
```

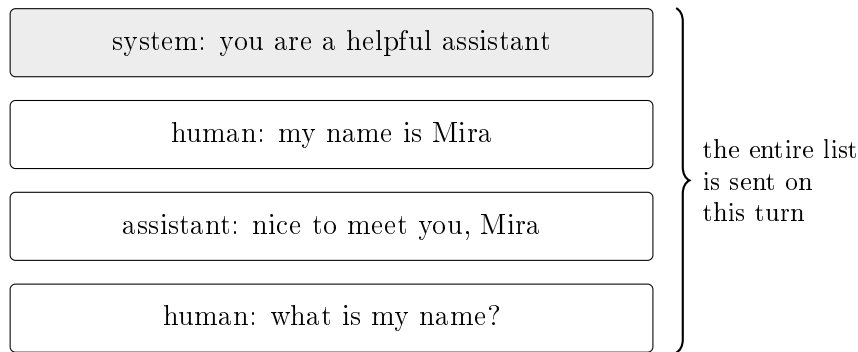


Figure 6.1: Short-term memory is created by resending the accumulated messages on every turn. The model has no store of its own; the growing list is the memory.

Listing 6.1: Short-term memory as a growing list of messages. Each turn appends to the history and sends all of it.

There is no hidden mechanism here, and that deserves a pause. The assistant that seems to remember is doing nothing more than reading a transcript that the application keeps handing back to it. The memory lives in the application’s list, not in the model.

6.3 What this memory can and cannot do

Because the history is present in the model’s context, the assistant handles the ordinary business of conversation well. Ask it, several turns after introducing yourself, whether it knows anything about you, and it will summarize what you said, because that information is sitting in the messages it was just given. Reference an earlier answer and it follows, because the earlier answer is there to be read. Within a single session this simple pattern is enough to make an assistant feel coherent.

Two limits deserve mention. The first is that the memory is short-term in a literal sense: it exists only while the application keeps the list. Close the session and the list is gone, and a new session starts blank. Memory that persists across sessions, so that the assistant recalls you tomorrow, is a separate problem requiring storage outside the conversation, and it is not solved by resending alone. The second limit is practical: the list cannot grow without bound, because a model can attend to only so much text at once, and every resent turn is text the model must process again. Long conversations eventually have to be trimmed or summarized, trading completeness of memory for room to continue.

6.4 Memory does not cure hallucination

There is a trap here that catches beginners, and it connects directly to Chapter 2. Because the assistant now recalls what the user said, it is tempting to conclude that it has become reliable. It has not. Memory governs whether the model can *see* the earlier turns; it says nothing about whether the model’s other statements are *true*.

The two behaviors can appear in the same exchange. Imagine a user who says early on that they live in a particular town, and later asks the assistant what it knows about them. The assistant correctly reports the town, because that fact is in the recent history, memory working as intended. In the same breath it may add a confident claim about which state or district the town belongs to, get it wrong, and, when challenged, produce a fresh and equally confident correction that is also wrong. The recall is faithful; the geography is invented. What the user supplied, the model can repeat; what the user did not supply, the model fills in with probable-sounding text, exactly as (2.3) would lead one to expect.

This is the clearest possible motivation for the remainder of the book. Conversational memory makes an assistant coherent, but coherence is not accuracy. To make the assistant trustworthy on matters of fact, particularly facts contained in the user's own documents, we need a way to place the right information in front of the model at the moment it answers, so that it repeats truth rather than inventing it. Building that mechanism is the work of Part III. First, though, we assemble the components we already have into clean pipelines, which is the subject of the next two chapters.

Exercises

1. Explain, in terms of the model being stateless, why an assistant that resends nothing appears to have no memory.
2. The chapter distinguishes short-term memory from memory that persists across sessions. Describe an application where the second is necessary and say what extra machinery it would require.
3. Construct a short imagined exchange in which an assistant shows working memory and hallucination at the same time, and identify which statements are which.

Chapter 7

Chains

By now the application has three kinds of part: a prompt that shapes the input, a model that produces text, and, as we are about to add, a parser that tidies the output. On their own each is invoked separately, and the code that carries the result of one into the next by hand becomes noise. A *chain* removes that noise. It connects components so that the output of one becomes the input of the next, and the whole sequence is then run as a single unit.

7.1 Components in a line

The commonest chain in an LLM application has three components, shown in Figure 7.1. A prompt template turns the user’s input into a well-formed prompt. The model turns that prompt into a reply. A parser turns the model’s reply into whatever form the rest of the program wants, most often plain text pulled out of the message object the model returns. Read left to right, the data flows straight through: input in one end, a clean answer out the other.

The output parser is the one new piece. A model returns a message object with several fields, not a bare string, and application code usually wants the string. A parser handles this small but constant chore, and more elaborate parsers can enforce a particular structure on the output, a point we return to when we discuss agents. For now it is enough to think of it as the component that extracts the answer from the model’s reply in the form the program needs.

7.2 Connecting components with LCEL

In current LangChain, chains are written using the LangChain Expression Language, or LCEL. Its central piece of notation is the pipe operator, the vertical bar `|`, borrowed from the shell, where it likewise means send the output of the thing on the left into the thing on the right. A three-component chain is written by piping the components together in order.

```
from langchain_core.output_parsers import StrOutputParser

chain = chat_template | model | StrOutputParser()
result = chain.invoke({"topic": "retrieval-augmented generation"})
print(result)
```

Listing 7.1: A chain built with the pipe operator. Reading left to right mirrors the flow of data through the components.

Two things about this deserve attention. The first is that the whole chain is run by the same `invoke` verb used for a single model, and takes the same kind of input, the placeholder values, as the prompt template alone would. The chain behaves, from the caller’s side, like

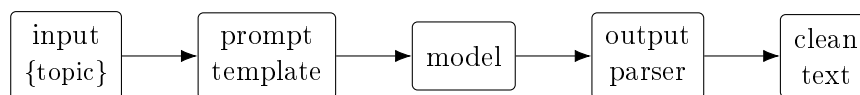


Figure 7.1: A three-component chain. Each arrow is a hand-off the chain performs automatically: the prompt’s output is the model’s input, and the model’s output is the parser’s input.

one component. This is not a coincidence of naming; it is the design principle that the next chapter makes explicit. The second is that the hand-offs between components are automatic. The engineer does not write code to pass the prompt’s messages to the model or the model’s reply to the parser; assembling the chain with `|` arranges for the output of each compatible component to flow into the next.

7.3 Why this is more than convenience

It would be fair to ask whether a chain is anything more than a tidy way of writing three calls. On a chain this short, the saving is modest. The value grows with the application. Because a chain is itself a runnable unit that speaks the same interface as its parts, a chain can be a component of a larger chain, and pieces can be swapped without disturbing what surrounds them: change the model and the prompt and parser are untouched; change the parser and the model does not notice. When we reach retrieval, the retrieval step will slot into a chain exactly as the prompt and model do, and the assistant will be a chain that happens to have a document-lookup component in the middle of it. Writing pipelines this way is what keeps them from collapsing into tangle as they grow.

LangChain did not remove the older, more verbose ways of building chains; they remain for compatibility. But the pipe-operator style is the modern and recommended one, and it is the style used for the rest of the book. What makes it work, the reason arbitrary components can be joined with a bar and simply fit, is that they all share one interface. That shared interface has a name, and it is the subject of the next chapter.

Exercises

1. Draw the data-flow for a chain that takes a topic, asks the model for a definition, and returns plain text. Label what type of value travels along each arrow.
2. Explain what an output parser does and why application code usually needs one after a model.
3. The chapter says a chain “behaves like one component.” Give one practical consequence of that property for building a larger application.

Chapter 8

Runnables: The Common Interface

The previous chapter left one question open: why can a prompt, a model, and a parser be joined with a pipe and simply work together, when they do quite different jobs? The answer is that LangChain gives all of them the same interface. Any component that follows this interface is called a *Runnable*, and once one sees the idea, a good deal of the framework falls into place at once.

8.1 One interface, three verbs

A Runnable is a component that follows a common input-to-output contract, which is what lets LangChain connect it to other components without knowing what it does inside. The contract is small. Every Runnable offers the same handful of methods, of which three matter most. **invoke** runs the component on a single input and returns a single output; this is the verb used in every previous chapter. **batch** runs the component on a list of inputs and returns a list of outputs, which a good implementation can process together for efficiency. **stream** runs the component and yields the output in pieces as it becomes available, which is how a chatbot can begin displaying an answer before the model has finished producing it. Figure 8.1 shows this shared interface.

Because every component answers to the same verbs, code that drives one Runnable drives any Runnable. That is the whole trick. A model is a Runnable, a prompt template is a Runnable, a parser is a Runnable, and, as the last chapter’s chain showed, they can be run identically because they share this contract.

8.2 You have been using Runnables already

The pipe operator of the previous chapter was Runnables at work, though nothing announced it. When components are joined with `|`, the result is a particular kind of Runnable called a *RunnableSequence*: a Runnable that holds an ordered list of Runnables and, when invoked, feeds each one’s output to the next. The chain `prompt | model | parser` is a *RunnableSequence* with three stages. Invoking the sequence invokes the stages in turn. This is why a chain behaves like a single component: it *is* a single component, one whose behavior is to run its parts in order.

Seen this way, the framework is more uniform than it first appears. There is really one kind of thing, a Runnable, and the various named classes are Runnables that do particular jobs.

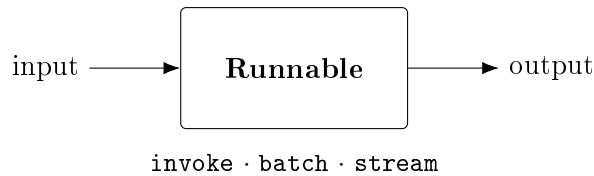


Figure 8.1: Every Runnable exposes the same interface: one input in, one output out, reachable through `invoke`, `batch`, or `stream`. Components differ in what they compute, not in how they are called.

8.3 Turning ordinary functions into Runnables

Not every step in a pipeline is a prompt or a model. Often a step is a plain piece of Python: clean up some text, pick a field out of a dictionary, reshape a value so the next component can read it. To place such a step inside a chain it must first become a Runnable, and `RunnableLambda` does exactly that, wrapping an ordinary function so that it gains the common interface.

```

from langchain_core.runnables import RunnableLambda

def word_count(text: str) -> str:
    return f"{text}\n\n(words: {len(text.split())})"

counter = RunnableLambda(word_count)
chain = chat_template | model | StrOutputParser() | counter
  
```

Listing 8.1: Wrapping an ordinary function as a Runnable so that it can join a chain. Once wrapped, it responds to `invoke` like any other component.

The wrapped function now sits in the pipe alongside the model and the parser, indistinguishable from them as far as the chain is concerned. This is how application-specific logic is folded into a pipeline without breaking its uniformity: anything can join a chain once it is a Runnable, and any function can be made a Runnable.

8.4 Chains that are not straight lines

So far every chain has been a straight line, each step feeding the one after it. Real applications are not always linear. Sometimes the same input should be sent to several components at once and their results combined; sometimes the path should branch depending on the input. Runnables that arrange other Runnables in these non-linear ways are called *primitives*. A primitive does no task of its own. Its job is to orchestrate: to say how a group of Runnables should be run relative to one another.

The most useful primitive after the sequence is the parallel map, `RunnableParallel` (also seen as `RunnableMap`), which runs several Runnables on the same input at once and gathers their outputs into a labeled dictionary. Figure 8.2 contrasts it with a sequence.

```

from langchain_core.runnables import RunnableParallel

analysis = RunnableParallel(
  
```

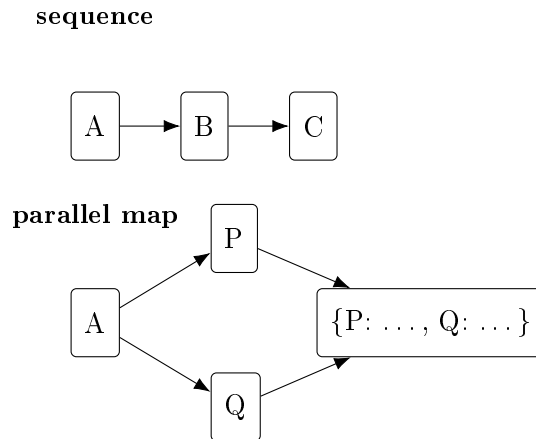


Figure 8.2: Two ways to arrange Runnables. A sequence passes one output to the next input; a parallel map sends one input to several Runnables and collects their outputs together.

```

summary=(summary_prompt | model | StrOutputParser()),
keywords=(keyword_prompt | model | StrOutputParser()),
)
result = analysis.invoke({"topic": "vector databases"})
# result -> {"summary": ..., "keywords": ...}

```

Listing 8.2: A parallel map runs several Runnables on the same input and returns their outputs together.

With sequences to run steps in order, parallel maps to run them side by side, lambdas to admit ordinary code, and a shared interface tying them together, the pieces needed for a substantial pipeline are in place. LangChain retained its older chain-construction methods for backward compatibility, but Runnables composed with these primitives are the current and recommended way to build. When the assistant is finally assembled in Part III, it will be a composition of exactly these parts: a retrieval step and a prompt run to prepare context, then a model and a parser to produce the answer, joined by the same operators used here.

Exercises

1. State the three principal Runnable methods and describe a situation in which each is the right one to call.
2. Explain why `prompt | model | parser` can be treated as a single component, referring to what a `RunnableSequence` is.
3. You need a chain step that lowercases its input text. Sketch how you would make this step a Runnable and where it would sit in a chain.
4. Give an application that benefits from a parallel map, and say what the labeled outputs would be.

Chapter 9

From Chatbots to Agents

Everything built so far produces text for a person to read. That is the natural output of a language model, and for a chatbot it is enough. A more capable class of application asks the model not only to talk but to act, to look something up, call a service, run a calculation, or change something in the world. Systems of this kind are called *agents*, and reaching them requires one further idea: getting the model to produce output that other software can use, not just output that a person can read.

9.1 Talking versus acting

The difference between a chatbot and an agent is a difference in who, or what, the system communicates with. A chatbot communicates with a user: text in, text out, a conversation. An agent communicates with a user *and* with tools, external services, and other software. Where the chatbot answers a question about the weather from whatever it happens to know, an agent calls a weather service, reads the reply, and answers from real data. The shift is from a system that speaks to one that also does. Figure 9.1 contrasts the two.

9.2 Software needs structure

A person can read a paragraph and extract what they need. A piece of software cannot. If an agent is to call a weather service, it must hand that service a request in a precise, agreed-upon form: the name of the city in one labeled place, the units in another, nothing ambiguous. Free-flowing prose, the model's usual output, is useless for this. For the model to drive software, its output has to be *structured*, arranged in fields with known names and predictable types, so that a program can pick out each value without guessing.

This is the real prerequisite for agents, and the point is simple enough to state plainly: to act through software, the model must speak the language of software, which is structured data. The rest is arranging for the model to produce that structure reliably.

9.3 Two kinds of output

The distinction between the model's two possible output styles is the crux of the chapter. *Unstructured* output is ordinary text meant for a human: a paragraph, an explanation, an answer. *Structured* output is data meant for a machine: labeled fields with definite values. A summary of a report is unstructured; the same report reduced to a record with a title field, a date field,

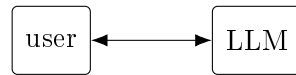
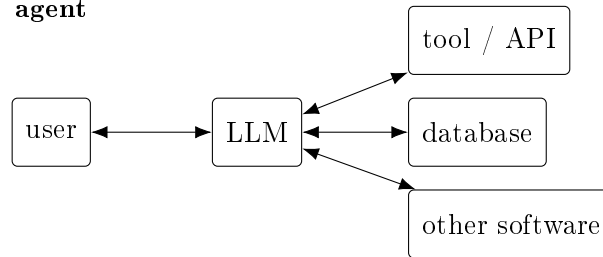
chatbot**agent**

Figure 9.1: A chatbot exchanges text with a user. An agent also exchanges data with tools, services, and other software, which is what lets it act rather than only answer.

and a list of action items is structured. The information may overlap, but only the second form can be fed to another program without a human in the loop.

The usual format for this structured data is JSON, short for JavaScript Object Notation. It is a lightweight, text-based format for storing and sending data between systems, readable by almost every programming language, which is why it has become the common tongue for software talking to software. A JSON record looks much like the labeled dictionary a parallel map produced in the last chapter, and the resemblance is not accidental.

```

# unstructured (for a person)
"The meeting is with Acme on 3 March about the annual renewal."

# structured (for software)
{"company": "Acme", "date": "2025-03-03", "topic": "annual renewal"}
  
```

Listing 9.1: The same information as unstructured text and as structured JSON. Only the second can be consumed directly by another program.

It helps to be clear about one point that confuses newcomers, because JSON resembles a Python dictionary closely enough to be mistaken for it. A Python dictionary is an in-memory data structure that a running program uses to hold and manipulate values. JSON is a text format used to store data or move it between systems. One lives inside a single program; the other travels between programs. An application routinely converts between the two, reading JSON text into a dictionary to work with it and writing a dictionary back out to JSON to send it on, but they are not the same thing, and keeping the distinction straight prevents a class of avoidable errors.

9.4 Getting structure out of the model

Asking a model politely for JSON in the prompt works some of the time and fails the rest, which for an agent is not good enough: a single malformed reply breaks the program downstream. The

practical approach is to specify the desired structure to the model as a schema, the fields and their types, and to use an output parser that both instructs the model and validates what comes back, rejecting or repairing output that does not conform. This is the more elaborate role of the output parser mentioned in Chapter 7. With a schema in place, the model’s reply can be trusted to have the agreed shape, and the value in each field can be read out and handed to whatever tool or service needs it.

Reliable structured output is what makes the high-value uses of these systems possible: pulling defined fields out of invoices and contracts, converting a request written in plain language into a precise call to an internal service, routing a message by reading its content into a category. Each is a case of a model reading unstructured input and emitting structured output that software then acts on. The agent, in the fullest sense, is a model that does this in a loop, deciding which tool to call, calling it, reading the result, and deciding what to do next.

Agents are where much of the current excitement in the field sits, and a serious treatment of planning, tool selection, and control would fill a book of its own. For our assistant we need only the groundwork this chapter has laid, the idea that a model can produce data as well as prose. The particular capability we now turn to, letting the assistant answer from documents it was never trained on, does not require full agency, but it does rest on the same habit of mind: deciding what information the model needs and arranging for it to be there. That is retrieval, and it is the subject of the final part.

Exercises

1. State the difference between a chatbot and an agent in terms of what each communicates with, and give one task that requires an agent.
2. Explain why an agent needs structured output rather than prose, using the example of calling an external service.
3. Distinguish a Python dictionary from JSON. Describe a point in an application where a conversion from one to the other takes place.
4. Take an everyday document, such as a receipt, and design a small JSON schema for the fields a program would want to extract from it.

Part III

Retrieval-Augmented Generation

This part solves the problem that has been named but left open since Part I: a model does not know your documents, and asked about them it invents fluent, confident, wrong answers. The remedy is to find the passages relevant to a question and place them in front of the model before it answers. The chapters build that remedy in order, first stating the knowledge problem exactly and ruling out the tempting shortcuts, then setting out retrieval-augmented generation as the pattern that answers it, then supplying the two pieces it needs, a way to measure relevance by meaning through embeddings, and a way to index a large collection through chunking and a vector store. The final chapter assembles everything, from Part II and this one, into the complete document-aware assistant that the book has been working toward. This is where the assistant stops merely sounding knowledgeable and becomes, on the material you give it, reliable.

Chapter 10

The Knowledge Problem

The assistant built up to this point can converse, remember a session, and produce structured output. It has a limitation that no amount of prompting fixes, and that limitation is the reason this last part of the book exists. The model knows a great deal in general and nothing at all about your particular documents, and for many real applications the particular documents are the whole point.

10.1 A model does not know your documents

A language model’s knowledge is whatever was present, implicitly, in the text it was trained on. That training set is vast, so the model can discuss history, explain physics, and write code in a dozen languages. It was not trained on your company’s internal handbook, your lab’s unpublished results, this year’s contracts, or the notes you took last week, because those were not in the training data and, in most cases, could not have been. Ask the model a general question and it answers from its training. Ask it a specific question whose answer lives only in a private document, and it has nothing to draw on.

What it does in that situation is the trap identified in Chapter 2. It does not say it does not know. It produces fluent, confident text that continues the prompt plausibly, and that text is, as often as not, wrong. The failure is quiet, which makes it dangerous: the answer looks exactly like a correct one. An assistant meant to answer questions about material it was never trained on cannot be built by making the model try harder. The missing information has to be supplied.

10.2 Two tempting non-solutions

Before giving the right answer, two wrong ones should be cleared away, because both occur to almost everyone.

The first is to train the model on the documents. This is possible in principle and is occasionally the right choice, but for most applications it is the wrong tool. Training, even the lighter form called fine-tuning, is costly, slow, and awkward to keep current: every time a document changes, the model is out of date, and retraining to fix a single updated figure is absurd. Training also teaches the model a general style or skill far better than it teaches it specific facts, and facts are exactly what we need it to get right.

The second is to paste every document into the prompt. If the model answers from whatever is in its context, why not put everything there? Because there is a hard limit on how much text a model can attend to at once, and a real document collection dwarfs it. Even where everything

would fit, sending an entire library on every question is wasteful and, as the collection grows, impossible. Stuffing the prompt does not scale.

10.3 What is actually needed

The right framing is more modest than either non-solution, and once stated it is almost obvious. When a specific question is asked, we do not need the model to have memorized the documents, and we do not need to give it all of them. We need to give it *the relevant part*, and only that part, at the moment it answers. If the passage that contains the answer is placed in the prompt, the model can read it and respond from it, repeating what the passage says rather than inventing something. The model's fluency, which was a liability when it had nothing to go on, becomes an asset when it has the right text in front of it.

So the problem reduces to a mechanism: given a question, find the small number of passages from the collection that bear on it, and put those passages into the prompt before the model answers. Two sub-problems sit inside this. We must be able to judge which passages are relevant to a question, which is harder than matching words, since a passage can answer a question without sharing its vocabulary. And we must be able to do this quickly over a collection far too large to scan in full on every query. The next chapter names the pattern that packages this mechanism, and the two after it solve the sub-problems, relevance by way of embeddings, and speed by way of chunking and a vector store. Together they turn the assistant from one that sounds knowledgeable into one that is, on the material we give it, actually reliable.

Exercises

1. Explain why a language model gives a confident wrong answer to a question about a private document, rather than admitting ignorance.
2. Give one situation in which fine-tuning a model on documents is reasonable and one in which it is clearly the wrong approach, and state what distinguishes them.
3. The chapter rejects pasting all documents into the prompt. State the two reasons and say which one applies even when the collection is small.

Chapter 11

Retrieval-Augmented Generation

The mechanism sketched at the end of the last chapter, find the relevant passages and place them in the prompt, has a name. It is called *retrieval-augmented generation*, usually shortened to RAG, and it is the standard way to make a language model answer from a body of documents it was never trained on. The idea is simple enough to state in a line and general enough to underlie a large share of the useful LLM applications now in service.

11.1 What RAG is, and is not

RAG has three movements, and its name is a fair summary of them. *Retrieve* the information relevant to the question from a collection of documents. *Augment* the prompt by adding that information to it. *Generate* the answer with the model, now reading the added information as context. The model does the same next-token generation it always did; the difference is entirely in what it is given to work from.

The name invites two misreadings, so it helps to be precise about what RAG is not. RAG is not a new kind of model. The language model at its center is an ordinary one, used unchanged. And RAG is not a way of training a model. Nothing is learned, no parameters are adjusted, no retraining takes place. RAG is a way of *giving a model the right information before asking it to answer*, and that is all. It sits entirely outside the model, in the application, which is exactly why it is the province of the application engineer rather than the model developer.

This placement is what makes RAG attractive. Because it changes nothing inside the model, it inherits none of training's costs. When a document changes, the collection is updated and the next query sees the change immediately; there is no model to bring up to date. The same application works with a different model by swapping the model out, since the retrieval machinery does not depend on which model reads the retrieved text. And because the answer is drawn from specific retrieved passages, the application can show *which* passages it used, giving the user a way to check the answer against its source, something a bare model cannot offer.

11.2 Two phases

A RAG system does its work in two phases, separated in time, shown together in Figure 11.1.

The first phase happens once, ahead of any question, and prepares the collection. Each document is broken into passages of a manageable size, a step called *chunking* (Chapter 13). Every passage is converted into a numerical form that captures its meaning, an *embedding* (Chapter 12), and the passages and their embeddings are stored together in a structure built for fast similarity search, a *vector store*. This phase is often called indexing. Once done, it need not

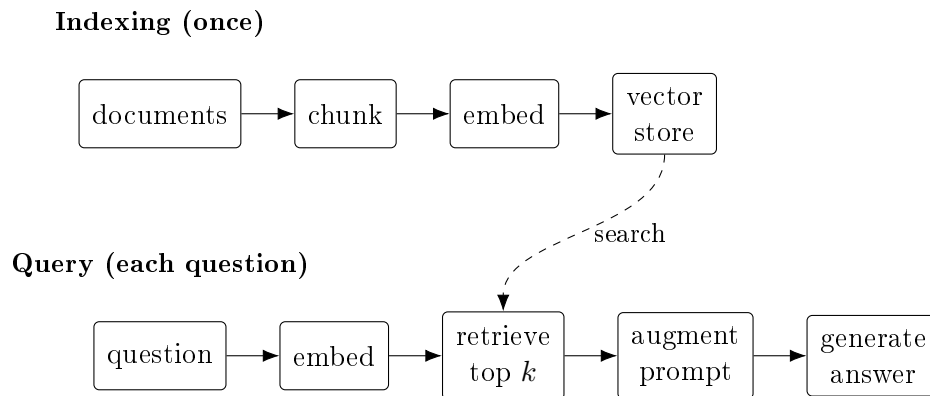


Figure 11.1: A RAG system in two phases. Indexing prepares the collection once; the query phase retrieves the relevant passages for each question and lets the model generate an answer from them.

be repeated unless the documents change.

The second phase happens on every question. The user’s question is converted into an embedding by the same method used for the passages. The vector store is searched for the passages whose embeddings are most similar to the question’s, and the best few are returned. Those passages are inserted into the prompt alongside the question, and the model generates an answer from them. This is the retrieve-augment-generate cycle proper, and it runs anew for each query.

11.3 Where the difficulty lies

Stated this way, RAG has one component that is genuinely subtle and several that are routine. Chunking, storage, and prompt construction are engineering: fiddly, consequential, but not deep. The subtle part is retrieval, and specifically the notion of similarity it depends on. When the question and the passage that answers it share no words, matching on words fails. A question about “how a transistor amplifies” should retrieve a passage on “a small base current controlling a larger collector current” even though the two have almost no vocabulary in common. Retrieval must compare *meaning*, not spelling.

Making meaning comparable by a computer is the problem the next chapter solves. We need a way to turn a piece of text into a form on which similarity can be measured numerically, so that the passage closest in meaning to a question can be found by calculation rather than by keyword. That form is the embedding, and the measure of closeness is a piece of simple geometry. Together they turn the vague word “relevant” into a number a machine can rank.

Exercises

1. Write the three movements of RAG in your own words and identify, for a question-answering assistant, what plays the role of each.
2. The chapter insists RAG is “not a way of training a model.” Explain the practical consequences of this for keeping an assistant’s knowledge current.
3. Separate the following steps into the indexing phase and the query phase: embedding the

question; chunking the documents; searching the vector store; storing passage embeddings.

4. Give a question-and-passage pair that share their meaning but almost none of their words, and explain why keyword matching would miss it.

Chapter 12

Embeddings and Semantic Similarity

Retrieval hinges on measuring how close in meaning two pieces of text are. A computer cannot compare meanings directly, but it can compare numbers, and it can do so at enormous speed. The task, then, is to represent the meaning of a piece of text as numbers in a way that preserves closeness: texts with similar meaning should map to numbers that are near each other, and unrelated texts to numbers that are far apart. The representation that does this is called an *embedding*, and it is the idea on which retrieval, and much else in modern natural language processing, rests.

12.1 Meaning as a vector

An embedding is a function that takes a piece of text and returns a vector, a list of d real numbers, which we write as a point in $\mathbb{R}^d$. Call this function E , so that for a text s ,

$$E(s) = \mathbf{e} \in \mathbb{R}^d.$$

The number d is the dimension of the embedding, fixed by the model that produces it and typically a few hundred. The individual numbers in $\mathbf{e}$ have no separate meaning one could name; what carries meaning is the position of the whole vector relative to others. The design goal of the embedding model is that this position should reflect meaning: two texts that say similar things land close together in $\mathbb{R}^d$, and two that say different things land far apart. When that holds, a geometric question, which vectors are near this one, becomes a stand-in for a semantic question, which texts mean something like this one.

How an embedding model comes to have this property is a matter of training, and we take it as given here in keeping with the book's division of labor: model developers train embedding models, and application engineers use them. A widely used one, small enough to run locally and good enough for most purposes, is `all-MiniLM-L6-v2` from the sentence-transformers family. It maps a piece of text to a vector of modest dimension, and it is the kind of model our assistant would use for indexing.

12.2 Words versus sentences

There is an important choice in *what* is embedded. Early work embedded individual words, giving each word a vector, and word embeddings remain useful for studying relationships between words. Retrieval needs something else. A question is not a word, and neither is the passage that answers it; each is a complete unit of meaning made of many words in a particular arrangement. What

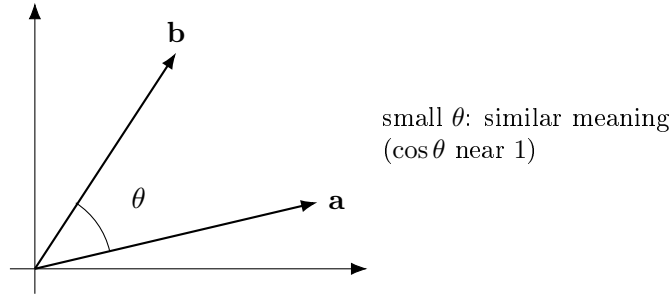


Figure 12.1: Cosine similarity measures the angle θ between two embedding vectors, ignoring their lengths. A small angle means the texts point the same way in the embedding space and so are close in meaning.

we want to compare are whole pieces of meaning, and for that we need *sentence embeddings*, which represent an entire sentence or passage as one vector.

The distinction matters because meaning is not simply the sum of the words present. “The dog chased the cat” and “the cat chased the dog” contain the same words and mean different things; a bare word-level representation cannot tell them apart, while a sentence embedding, trained to capture the meaning of the whole, can. Since RAG compares questions with passages, both complete pieces of meaning, it is sentence embeddings that it uses. The sentence-transformers models named above are built for exactly this: to represent complete pieces of meaning rather than isolated words.

12.3 Measuring closeness by angle

With texts represented as vectors, closeness becomes a geometric quantity. The measure used almost universally in retrieval is *cosine similarity*, which judges how alike two vectors are by the angle between them rather than by their lengths. For two vectors $\mathbf{a}$ and $\mathbf{b}$ in $\mathbb{R}^d$ with angle θ between them, cosine similarity is

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \cos \theta = \frac{\langle \mathbf{a}, \mathbf{b} \rangle}{\|\mathbf{a}\| \|\mathbf{b}\|} = \frac{\sum_{i=1}^d a_i b_i}{\sqrt{\sum_{i=1}^d a_i^2} \sqrt{\sum_{i=1}^d b_i^2}}, \quad (12.1)$$

where a_i and b_i are the i th components of $\mathbf{a}$ and $\mathbf{b}$, the numerator $\langle \mathbf{a}, \mathbf{b} \rangle$ is their inner product, and $\|\mathbf{a}\|$ is the Euclidean length of $\mathbf{a}$. Dividing by the two lengths removes the effect of magnitude, so what remains depends only on direction. The value lies between -1 and 1 . A value near 1 means the vectors point in nearly the same direction, which for well-trained embeddings means the texts are close in meaning; a value near 0 means they are roughly unrelated; a value near -1 means they point in opposite directions. Figure 12.1 shows the geometry.

Using the angle rather than, say, the straight-line distance has a practical reason: it does not care how long the vectors are, only where they point, and for text embeddings direction carries the meaning while length is largely incidental. This is why a short question and a long passage

can be compared sensibly even though one produces a longer vector than the other.

12.4 Putting the two together

The pair of ideas in this chapter is the engine of retrieval, and a slogan captures it: embedding turns meaning into numbers, and cosine similarity tells us which meanings are close. In practice the two are used exactly as one would expect. Each passage in the collection is embedded once, during indexing. When a question arrives it is embedded by the same model, and its vector is compared by (12.1) against the stored passage vectors; the passages with the highest cosine similarity are the ones judged most relevant.

```
from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer("all-MiniLM-L6-v2")
q = model.encode("How does a transistor amplify a signal?")
p = model.encode("A small base current controls a larger collector current.")
score = util.cos_sim(q, p)    # a cosine similarity near 1 signals relevance
```

Listing 12.1: Embedding two texts and scoring their similarity. The same model must embed both the passages and the questions, or the comparison is meaningless.

One requirement is easy to overlook and important to get right: the questions and the passages must be embedded by the *same* model. Two different embedding models place text in two different spaces, and a cosine similarity computed across them is meaningless. This is a common early mistake and a quiet one, since the code runs and returns numbers; the numbers are simply nonsense.

What remains before a working system is a matter of scale and organization. Real collections hold far more passages than one would want to score one at a time on every query, and the documents arrive as long files rather than ready-made passages. The next chapter deals with both: how to cut documents into passages of the right size, and how to store their embeddings so that the most similar few can be found quickly.

Exercises

1. Explain why retrieval uses sentence embeddings rather than word embeddings, using an example where word order changes meaning.
2. From (12.1), explain what role the two length terms in the denominator play and why removing the effect of magnitude is desirable for comparing text.
3. Two texts have cosine similarity close to 0. What does this say about their meanings? What would a value close to 1 say?
4. Why must the same embedding model be used for questions and passages? Describe what goes wrong if two different models are used, and why the error is hard to notice.

Chapter 13

Chunking and Vector Stores

Two practical matters stand between the geometry of the last chapter and a working retrieval system. Documents do not arrive as tidy passages; they arrive as long files that must be cut into pieces before they can be embedded. And a real collection holds far too many pieces to compare one at a time on every question, so the pieces must be stored in a way that makes finding the closest few fast. This chapter handles both, and with them the indexing phase of a RAG system is complete.

13.1 Why documents are cut into chunks

A whole document is the wrong unit to embed. An embedding compresses a text into a single vector of fixed size, and the longer and more varied the text, the more that compression blurs. A vector standing for an entire manual represents everything in the manual and therefore nothing in particular; a question about one paragraph would match it only weakly, diluted by all the unrelated content. Beyond a certain length the text also exceeds what the embedding model can take in at once. For both reasons a document is divided into smaller pieces, called *chunks*, and each chunk is embedded on its own.

The size of a chunk is a genuine trade-off, and it is the central decision in chunking. If chunks are too large, each covers several topics and retrieval loses precision, since a chunk retrieved for one sentence drags along much that is irrelevant, and the model's prompt fills with noise. If chunks are too small, each may be cut off before it contains a complete thought, and a passage that has been split across two chunks may answer a question in neither. The objective, stated compactly, is to make chunks small enough for precise retrieval but large enough to preserve the meaning of the information they carry. There is no universal best size; it depends on the documents, and finding a good value is part of building the system.

13.2 Overlap

Cutting a document at fixed boundaries risks slicing through the middle of a sentence or an idea, leaving the two halves in separate chunks where neither is whole. The usual guard against this is *overlap*: consecutive chunks are made to share a stretch of text at their boundary, so that anything falling near a cut appears complete in at least one chunk. A common arrangement, and the one used in the project of the next chapter, is chunks of about a thousand characters with two hundred characters of overlap between neighbors.

The arithmetic can be stated once. Let each chunk have length L and let neighboring chunks

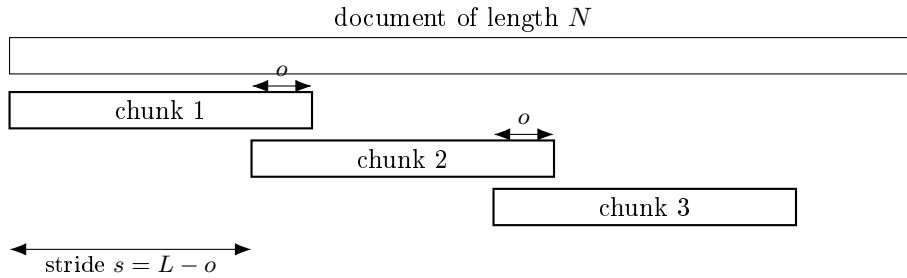


Figure 13.1: Chunking with overlap. Each chunk has length L ; neighbors share o characters, so each chunk begins a stride $s = L - o$ after the last. The overlap keeps material near a cut whole in at least one chunk.

overlap by o characters, with $0 \leq o < L$. Then each new chunk begins a *stride* of

$$s = L - o$$

characters past the previous one. For a document of length $N > L$, the number of chunks needed to cover it is

$$m = \left\lceil \frac{N - o}{L - o} \right\rceil, \quad (13.1)$$

which shows the cost of overlap directly: as o approaches L the denominator shrinks and m grows, so more overlap means more chunks, more embeddings to compute and store, and more redundancy. The overlap o is a hyperparameter, chosen by the engineer, and like the chunk length it is tuned rather than derived. Figure 13.1 shows the layout.

Cutting strictly by character count is the simplest scheme and the one the arithmetic above describes. More careful splitters prefer to cut at natural boundaries, between paragraphs or sentences, so that chunks end where meaning ends, falling back to a character limit only when a passage runs long. The principle is unchanged; only the choice of where to cut improves.

13.3 Storing embeddings for fast search

After chunking and embedding, the collection is a set of chunks, each paired with its embedding vector. Call the set of chunks $\mathcal{D}$. When a question q arrives, retrieval must find the chunks whose embeddings are most similar to the question's embedding $E(q)$. Written out, retrieval returns

$$\mathcal{R} = \text{TopK}_{c \in \mathcal{D}} \text{sim}(E(q), E(c)), \quad (13.2)$$

the k chunks c whose embeddings score highest on the cosine similarity of (12.1). The number k , how many passages to retrieve, is another hyperparameter, usually a small integer: enough to be likely to include the answer, few enough to keep the prompt focused.

Computing (13.2) by scoring every chunk against the query is fine for a small collection and too slow for a large one, where $|\mathcal{D}|$ may run to millions. A *vector store* is a data structure, and usually a piece of software around it, built for this one operation: it holds the chunk embeddings and, given a query vector, returns the most similar few quickly, using indexing methods that avoid comparing against everything. From the application's point of view a vector store offers

two operations, add a chunk with its embedding, and given a query embedding return the top k most similar chunks, and the details of how it achieves speed can be left to it.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
chunks = splitter.split_text(document_text)
# each chunk is embedded and stored; the store answers similarity queries
vector_store.add_texts(chunks)
```

Listing 13.1: The shape of indexing: split a document into overlapping chunks, then add them to a vector store that will embed and index them for search.

With chunking and a vector store in place, the indexing phase from Chapter 11 is fully specified: documents are split into overlapping chunks, each chunk is embedded, and the embeddings are placed in a vector store that supports fast top- k search. The query phase now has everything it needs, embed the question, retrieve by (13.2), add the retrieved chunks to the prompt, and generate. The next chapter assembles all of it into a single working application.

Exercises

1. Explain the two failure modes of poor chunk sizing, one for chunks that are too large and one for chunks that are too small, and relate each to retrieval quality.
2. Using (13.1), compute the number of chunks for a document of $N = 5000$ characters with $L = 1000$ and $o = 200$. Then recompute with $o = 0$ and comment on the difference.
3. State what problem overlap solves and what it costs, referring to how m in (13.1) depends on o .
4. Explain why a vector store is preferred over scoring (13.2) against every chunk, and name the two operations an application needs from it.

Chapter 14

Case Study: Chatting with a Video

Every idea in the book now comes together in one application. The project is deliberately concrete, so that each abstract piece, the prompt, the chain, the embedding, the chunk, the retrieval, has a visible job. The aim is a system that lets a user ask questions of a long video and get answers drawn from what the video actually says.

14.1 The problem

A great deal of useful material is published as long video: recorded lectures, technical talks, interviews, and podcasts that run for two or three hours. The format is poor for reference. To check one fact, settle one doubt, or find the ten minutes that matter, a viewer has to scrub through the whole thing, which is slow and discouraging. The information is present but not accessible, and the length of the video is exactly the obstacle.

14.2 The solution in outline

The plan is to build a RAG system over the content of a single video, so that a user can, in effect, hold a conversation with it: ask a question in plain language and receive an answer grounded in what was said, along with the ability to ask follow-ups. Nothing in this plan is new. It is the two-phase RAG pattern of Chapter 11 applied to one particular kind of document, where the document happens to be the spoken content of a video. Figure 14.1 shows the whole pipeline, and every box in it has appeared in an earlier chapter.

14.3 Getting the text out of the video

A video cannot be embedded directly, but its transcript can. Most videos carry a transcript, the spoken words as text with timing, and there is a straightforward way to obtain it. This is a good moment to be clear about what an *application programming interface* is, since the project depends on one. An API is an agreed way for one program to ask another for something. Here a small library, the YouTube transcript API, communicates with YouTube on our behalf: given the identifier of a video, it requests the transcript and returns the text, so that our Python program never has to deal with YouTube directly.

The transcript arrives as a series of short timed snippets rather than as continuous prose. For our purposes the timing is not needed, so the snippets are joined into one long string, which becomes the document that feeds the indexing phase.

```
from youtube_transcript_api import YouTubeTranscriptApi
```

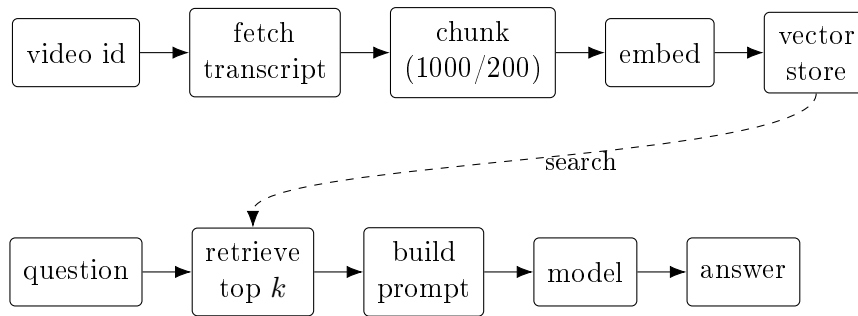


Figure 14.1: The capstone pipeline. The top row indexes the video’s transcript once; the bottom row answers each question by retrieving the relevant passages and letting the model generate from them.

```

video_id = "16ZcFa8pybE"
snippets = YouTubeTranscriptApi.get_transcript(video_id, languages=["en"])
transcript = " ".join(snippet["text"] for snippet in snippets)

```

Listing 14.1: Fetching a transcript and joining it into a single document. The API hides the details of talking to YouTube; the program receives text.

14.4 Indexing the transcript

With the transcript as a single string, the indexing phase is exactly Chapter 13. The transcript is split into overlapping chunks, a thousand characters each with two hundred characters of overlap, so that no idea is lost at a boundary. Each chunk is embedded with a sentence-transformers model, and the embeddings go into a vector store that will answer similarity queries. This work is done once per video; a second question about the same video reuses the index.

```

from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
chunks = splitter.split_text(transcript)
vector_store.add_texts(chunks)          # embeds and indexes each chunk
retriever = vector_store.as_retriever(search_kwargs={"k": 4})

```

Listing 14.2: Indexing the transcript once: split into overlapping chunks, embed, and store for search.

The retriever produced on the last line is the retrieval step of (13.2) wrapped as a component. Given a question it returns the four most similar chunks. It is, like everything else in LangChain, a Runnable, which is what lets it drop into a chain beside the prompt and the model.

14.5 Answering a question

The query phase assembles the parts from Part II around the retriever. A prompt template is written that takes two inputs, the retrieved context and the user’s question, and instructs the

model to answer from the context. This instruction is where the assistant's fourth requirement, not to invent, is enforced: the prompt tells the model to answer only from the provided passages and to say plainly when they do not contain the answer. The retriever, prompt, model, and output parser are then joined into a single chain with the pipe operator, and the whole thing is run with `invoke`.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableParallel, RunnablePassthrough

prompt = ChatPromptTemplate.from_messages([
    ("system",
     "Answer the question using only the context below. "
     "If the answer is not in the context, say you do not know.\n\n"
     "Context:\n{context}"),
    ("human", "{question}"),
])

def join(docs):
    return "\n\n".join(d.page_content for d in docs)

chain = (
    RunnableParallel(
        context=retriever | join,
        question=RunnablePassthrough(),
    )
    | prompt | model | StrOutputParser()
)

print(chain.invoke("What problem does the speaker say RAG solves?"))
```

Listing 14.3: The query phase as one chain. Retrieval, prompt construction, generation, and parsing are composed with the same operators used throughout Part II.

The parallel map at the top is doing something worth reading slowly. It sends the same question down two paths at once. On one path the question goes to the retriever, whose returned chunks are joined into a single block of context text. On the other path the question is passed through unchanged. The two results are gathered into a dictionary with the keys the prompt expects, `context` and `question`, which the prompt then fills. From there the chain is the familiar prompt-model-parser sequence of Chapter 7. The instruction to answer only from the context, together with retrieval that supplies the right context, is what keeps the assistant honest: when the transcript covers the question the model answers from it, and when it does not the model is told to say so rather than guess.

14.6 What has been built

The finished system is small, a few dozen lines, but it exercises the whole book. It runs a model (Part I), shapes its input with prompts and composes its parts into a chain of Runnables (Part II), and grounds its answers in retrieved passages so that it speaks from the document rather than from its own uncertain memory (Part III). The same skeleton, change the source of the documents and little else, serves a support assistant over a company’s manuals, a research aide over a folder of papers, or the document-aware personal assistant that has been the target since Chapter 3. The video was only a convenient document; the pattern is general.

Exercises

1. Match each stage of Figure 14.1 to the chapter that introduced it.
2. The prompt instructs the model to answer only from the context and otherwise to say it does not know. Explain how this instruction, combined with retrieval, addresses the hallucination problem of Chapter 2.
3. In the query chain, one path retrieves and one passes the question through. Explain why both are needed and what the parallel map produces.
4. Describe the changes required to turn this “chat with a video” system into a “chat with a folder of PDFs” system, and identify what stays the same.

A Closing Note: Where to Go Next

The assistant is built. Looking back, no step required inventing a model or deriving an algorithm; each took a capable component that already existed and gave it a job, and the engineering was in the arranging. That is the characteristic work of the application level, and it is where most of the value in this field is currently created. If a single thread runs through the book, it is the tension between fluency and truth: a language model is fluent by construction and truthful only by accident, and nearly every technique here exists to close that gap.

Several directions extend naturally from here, and any of them can be taken up without starting over. Retrieval has the most immediate depth: the simple top- k scheme of Chapter 13 can be sharpened by better chunking, by combining semantic similarity with keyword matching, and by re-ranking the retrieved passages before they reach the model. Agents, sketched in Chapter 9, open up once structured output is solid; an assistant that decides which tool to call and acts on the result is a large step beyond one that only answers, and it brings its own questions of control and safety. Persistent memory, the limitation flagged in Chapter 6, is worth taking up for any assistant meant to serve the same user across sessions, and it connects back to retrieval, since long-term memory is itself something to be searched. Underlying all of these is the ordinary craft of software: testing, handling errors, and watching how a system behaves once real users reach it.

The technology will keep moving, and particular libraries, model names, and interfaces will change. The ideas will not, or not as fast. A model predicts the next token and knows nothing of truth; meaning can be turned into vectors and closeness measured by angle; the way to make a model answer reliably is to place the right information in front of it. The specific commands in this book will date; the habit of thinking clearly about what a model needs, and arranging for it to be there, will not.